

PENETRATION TEST REPORT

Reflected Cross-Site Scripting (XSS) Assessment

Target Application	DVWA (Damn Vulnerable Web Application)
Target IP Address	10.114.186.145
Assessment Date	June 26, 2026
Assessed By	Mohamed Abuzahda
Tools Used	Firefox
Classification	Confidential — Educational Lab Report
Severity	HIGH

1. Executive Summary

A reflected Cross-Site Scripting (XSS) assessment was conducted against the DVWA (Damn Vulnerable Web Application) target hosted on TryHackMe at 10.114.186.145. The assessment covered all three security levels (Low, Medium, and High) of the XSS (Reflected) module.

Reflected XSS vulnerabilities were successfully identified and exploited at every security level. Arbitrary JavaScript was executed in the context of the application's origin at Low, Medium, and High security, demonstrating that user-supplied input is rendered back into the page without adequate sanitisation or output encoding. A proof-of-concept payload was also used to simulate session cookie exfiltration, illustrating how this class of vulnerability could be leveraged for session hijacking or account takeover in a production deployment.

Field	Detail
Target	10.114.186.145 — DVWA on TryHackMe
Module	XSS (Reflected)
Vulnerability	Reflected Cross-Site Scripting in the name parameter
Severity	HIGH
Security Levels Broken	Low, Medium, High
Impact	Arbitrary JavaScript execution; PoC session-cookie exfiltration
Assessment Period	June 26, 2026

2. Scope & Methodology

2.1 Scope

Application	DVWA v1.10 Development
Platform	TryHackMe (isolated lab environment)
Target URL	http://10.114.178.6/vulnerabilities/xss_r/
Server OS	Linux Ubuntu
Levels Tested	Low, Medium, High

2.2 Methodology

The assessment followed a structured exploitation methodology:

- Reflection Point Confirmation — submit a unique marker string in the name parameter and confirm it is echoed back into the HTML response without encoding.
- Low-Level Exploitation — inject a standard `<script>` payload to confirm unauthenticated JavaScript execution.
- Medium-Level Filter Bypass — analyse and bypass the `str_replace()` blacklist filter using case manipulation and alternative attack vectors.
- High-Level Filter Bypass — analyse and bypass the regex-based filter using non-`<script>` vectors such as event-handler attributes.
- Impact Validation — craft a proof-of-concept payload that exfiltrates `document.cookie` to demonstrate real-world session-hijacking risk.

3. Findings

3.1 Confirming the Reflection Point (Low Security)

Severity	High
Parameter	GET
Endpoint	/vulnerabilities/xss_r/?name=<payload>
Vector	Direct <script> tag injection
Filtering	None

Description

At Low security, the name parameter is concatenated directly into the page's HTML output with no encoding, filtering, or sanitisation applied. The vulnerable server-side logic is functionally equivalent .

Because the value is echoed verbatim inside the page body, any HTML or JavaScript supplied in the name parameter is parsed and executed by the victim's browser exactly as if it were part of the original page.

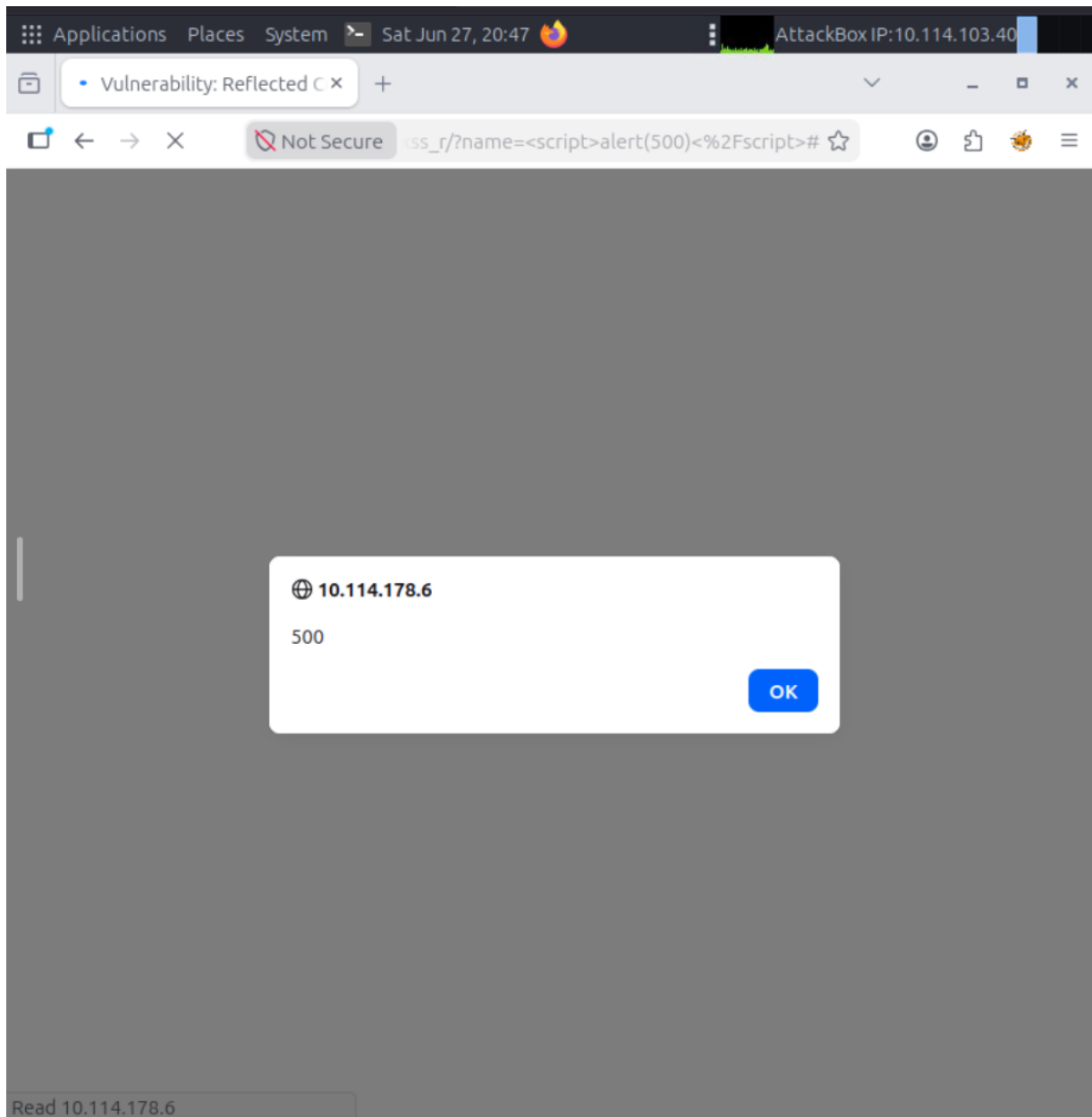
Proof-of-Concept Payload

```
<script>alert('500')</script>
```

Proof-of-Concept URL

```
http://10.114.178.6/vulnerabilities/xss_r/?name=<script>alert('500')</script>
```

Evidence — Figure 1: Reflected XSS Execution (Low Security)

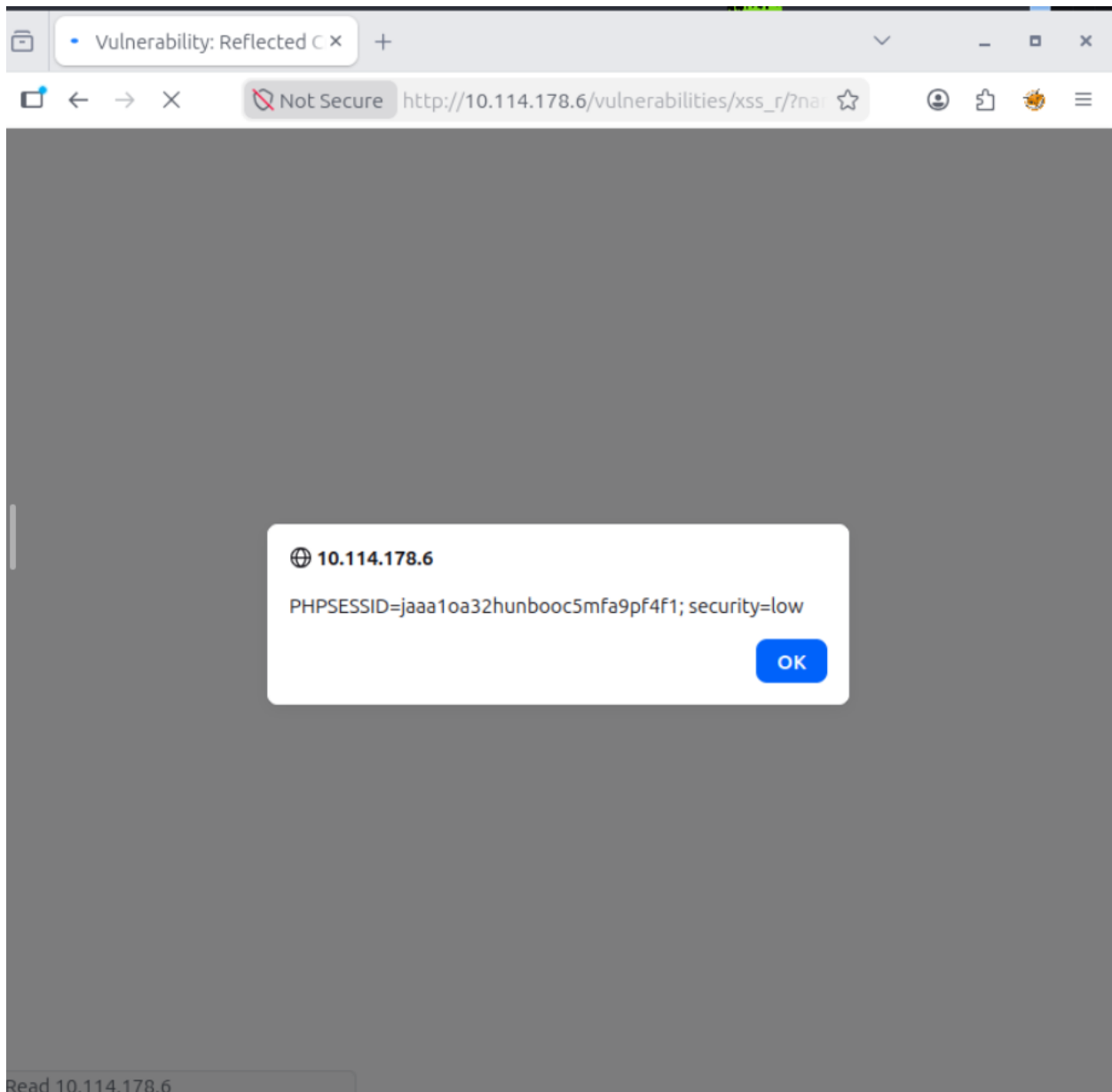


Impact Validation — Simulated Session Cookie Theft

To demonstrate real-world impact beyond a simple alert box, the following payload was used to show how an attacker could exfiltrate a victim's session cookie to an attacker-controlled server (a local listener was used in this lab in place of a real external host):

```
<script>alert(document.cookie)</script>
```

http://10.114.178.6/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28document.cookie%29%3C%2Fscript%3E#



In a live attack, this payload would be delivered via a crafted link (e.g. in a phishing email) and, when clicked by an authenticated victim, would silently send their PHPSESSID cookie to the attacker, enabling session hijacking without ever needing the victim's password.

3.2 Medium Security Level — Bypassing the str_replace() Filter

Security Level	Medium
Protection Added	str_replace('<script>', '', \$name) blacklist filter
Bypass Method	Case manipulation / non-script tag vector
Endpoint	/vulnerabilities/xss_r/?name=<payload>

Description

At Medium security, DVWA attempts to strip the exact, lowercase substring <script> from user input before reflecting it back into the page. The relevant logic is functionally equivalent .

This blacklist has two critical weaknesses: it is case-sensitive (so <SCRIPT> or <ScRiPt> is never matched and passes through untouched), and it only targets one specific tag name, leaving every other HTML element and event-handler attribute completely unfiltered. Either weakness alone is sufficient to bypass the filter.

Bypass Payload 1 — Case Manipulation

```
<scriPt>alert(document.cookie)</scriPt>
```

Bypass Payload 2 — Non-script Vector (Event Handler)

```
<img src=x onerror="alert('500')">
```

Both payloads were submitted via the name parameter and executed successfully, confirming the filter can be bypassed through either technique.

Proof-of-Concept URL

http://10.114.178.6/vulnerabilities/xss_r/?name=%3Cimg+src%3Dx+onerror%3Dalert%28500%29%3E#

Evidence — Figure 2: Filter Bypass Execution (Medium Security)

http://10.114.178.6/vulnerabilities/xss_r/?name=%3CscriPt%3Ealert%28document.cookie%29%3C%2FscriPt%3E#



- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)**
- XSS (Stored)
- CSP Bypass
- JavaScript
- DVWA Security
- PHP Info
- About

Vulnerability: Reflected Cross Site Scripting

What's your name?

Hello 

More Information

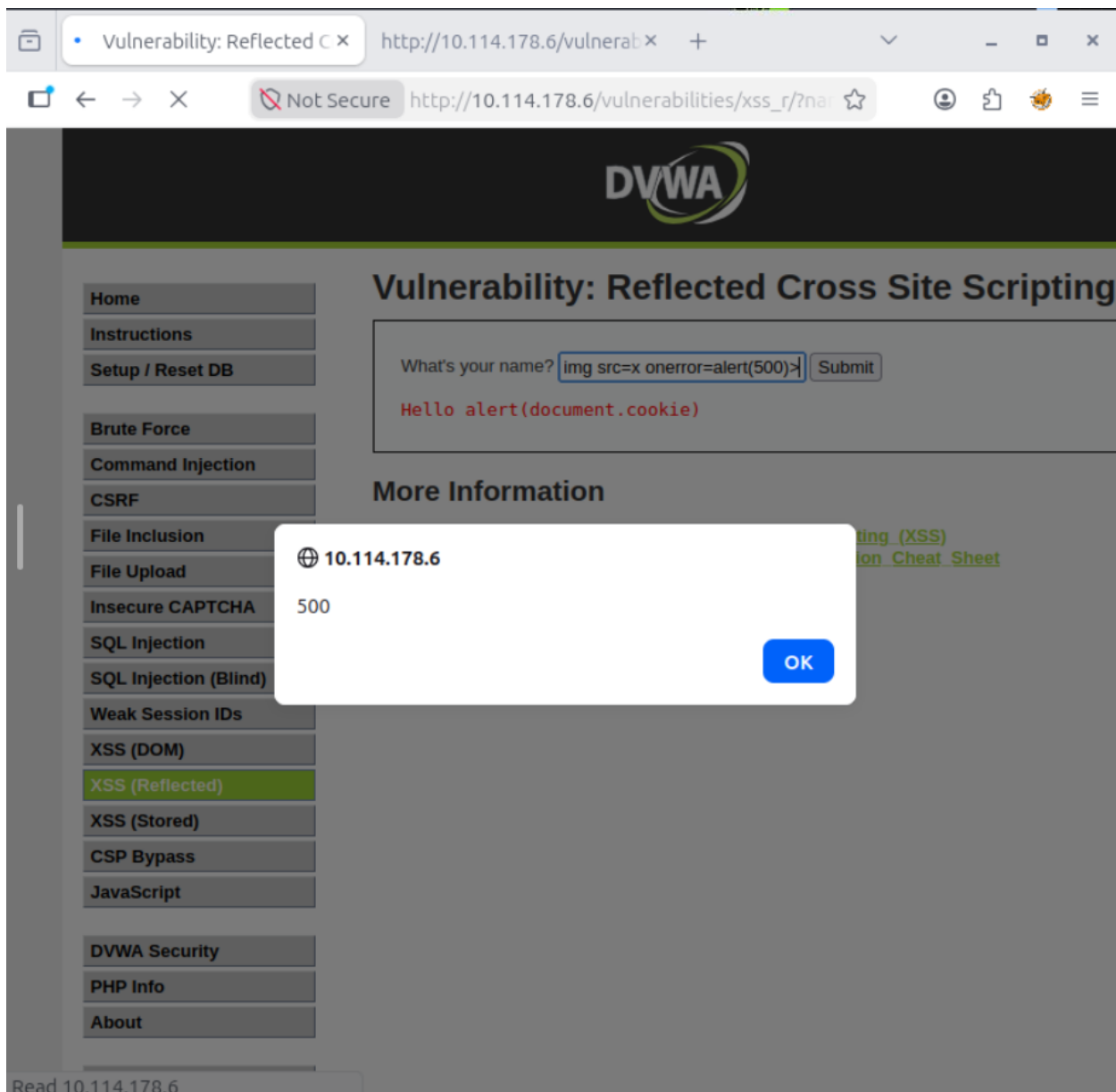
🌐 10.114.178.6

PHPSESSID=jaaa1oa32hunbooc5mfa9pf4f1;
security=medium

OK

[Reflected XSS](#)
[Reflected XSS Cheat Sheet](#)

Read 10.114.178.6



3.3 High Security Level — Bypassing the Regex Filter

Security Level	High
Protection Added	preg_replace() regex filter targeting <script> variants
Bypass Method	Non-script HTML vector with inline event handler
Endpoint	/vulnerabilities/xss_r/?name=<payload>

Description

At High security, DVWA replaces the simple blacklist with a case-insensitive regular expression intended to catch obfuscated variants of the script tag. The relevant logic is functionally equivalent .

This regex is considerably stronger against script-tag obfuscation, but it only matches tags that contain the literal letters s-c-r-i-p-t in order. It does not inspect any other tag name or attribute. Any payload that triggers JavaScript execution without using the word "script" — such as an HTML element with an inline event-handler attribute — bypasses the filter entirely.

Bypass Payload 1 — Image Error Handler

```
<img src=x onerror=alert('500')>
```

Bypass Payload 2 — SVG Load Handler (cookie exfiltration PoC)

```
<svg onload=alert(document.cookie)>
```

Proof-of-Concept URL

```
http://10.114.178.6/vulnerabilities/xss\_r/?name=<svg  
onload=alert\(document.cookie\)>
```

Both payloads executed successfully at High security, confirming that the regex filter can be fully bypassed using vectors that never spell out the word "script".

Evidence — Figure 3: Filter Bypass Execution (High Security)

Not Secure http://10.114.178.6/vulnerabilities/xss_r/?na



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Vulnerability: Reflected Cross Site Scripting

What's your name?

Hello

More Information

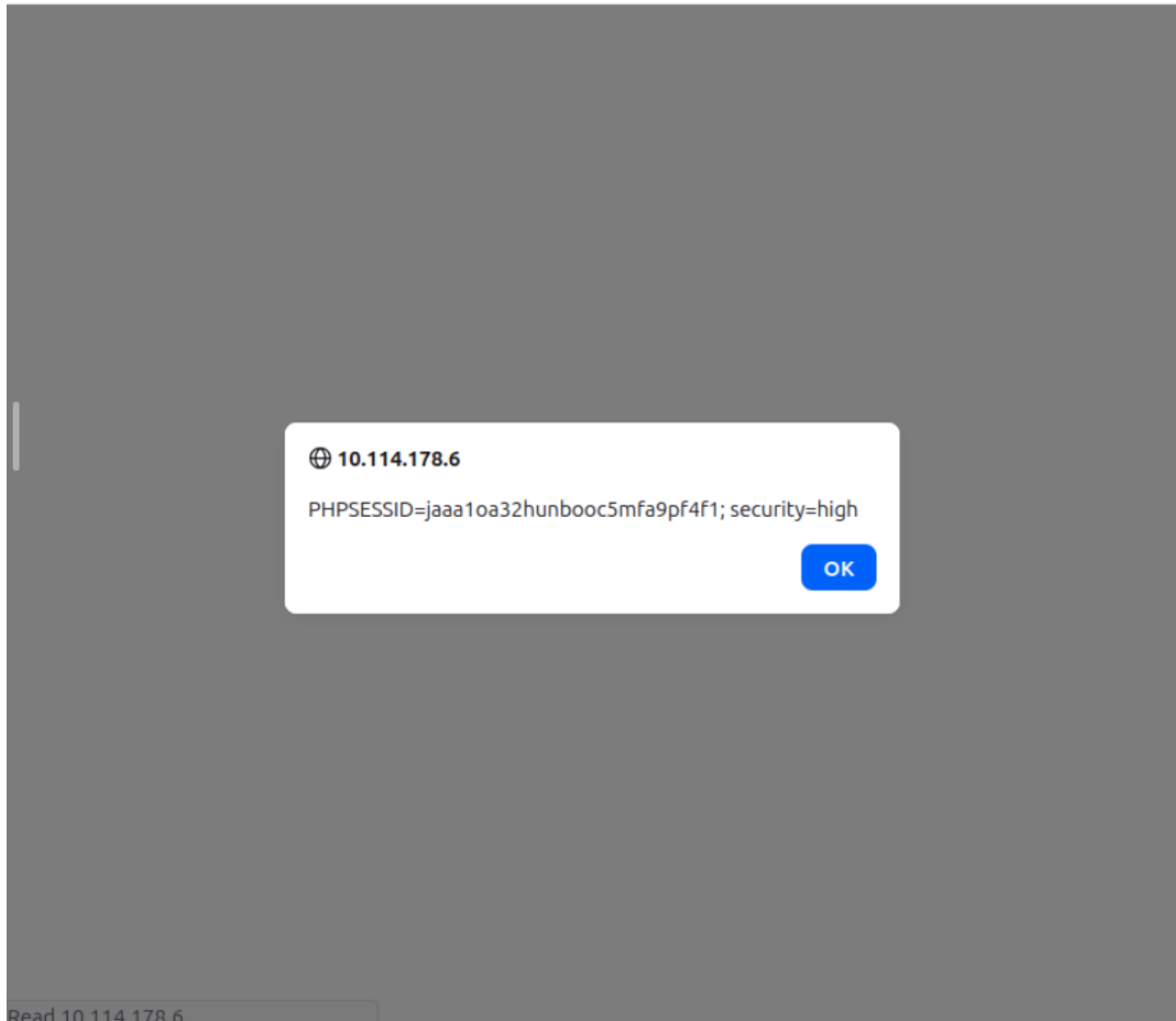
[Exploiting \(XSS\)](#)
[Exploitation Cheat Sheet](#)

10.114.178.6

500

OK

Transferring data from 10.114.178.6...



4. Recommendations

4.1 Apply Context-Aware Output Encoding

The root cause of every finding is that user-supplied input is written into the HTML response without encoding. All dynamic output must be encoded for the context in which it is rendered before being sent to the browser. For HTML body content, this means encoding special characters so they are displayed as text rather than parsed as markup:

```
// Vulnerable (current code):
echo '<pre>Hello ' . $_GET['name'] . '</pre>';

// Fixed (contextual output encoding):
echo '<pre>Hello ' . htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8') . '</pre>';
```

4.2 Replace Blacklisting with Allow-Listing

Both the Medium and High filters attempt to strip or reject known-bad patterns (`str_replace()` of a fixed string, and a regex targeting the word "script"). Blacklists are inherently incomplete because there are always more ways to trigger script execution than a filter author can anticipate. Output encoding (4.1) removes the need for any blacklist at all; where input validation is still desired, use a strict allow-list of expected characters (e.g. letters and spaces only) and reject anything else.

4.3 Implement a Content Security Policy (CSP)

A strict Content-Security-Policy header (e.g. restricting `script-src` to the application's own origin and disallowing inline scripts and `eval()`) provides a strong, defence-in-depth control that can prevent injected scripts from executing even if an output-encoding gap is later reintroduced.

4.4 Set HttpOnly and Secure Flags on Session Cookies

Marking session cookies as `HttpOnly` prevents JavaScript — including injected attacker scripts — from reading `document.cookie`, which would have blocked the cookie-theft proof-of-concept demonstrated in this report even if the underlying XSS still existed. The `Secure` flag should also be set to ensure cookies are only transmitted over HTTPS.

4.5 Deploy a Web Application Firewall (WAF)

A WAF adds a detection and blocking layer for common XSS patterns. While not a replacement for secure coding practices, a WAF provides an important additional layer of defence-in-depth.

4.6 Upgrade Server Components

PHP 5.5.9 has been end-of-life since July 2016 and contains numerous known vulnerabilities unrelated to this finding. Upgrade to a currently supported version of PHP (8.2+) and keep all server components patched.

Appendix A: Complete Payload Test Matrix

All payloads submitted during this assessment and their outcomes are reproduced below for full reproducibility.

Security Level	Payload	Result
XSS payload	<code><script>alert(500)</script></code> <code><scriPt>alert(500)</scriPt></code> <code></code>	Executed — alert box displayed
Low	<code><script>alert(document.cookie);</script></code>	Executed — PoC cookie exfiltration
Medium	<code><scriPt>alert(document.cookie)</scriPt></code>	Executed — PoC cookie exfiltration
High	<code><svg onload=alert(document.cookie)></code>	Executed — PoC cookie disclosure